

Discovery Piscine - AI/ML

Module 1 - Game Theory

Summary: Create a classic 15-puzzle game that automatically solves itself!

Version: 1.01

Contents

I	**Preamble**	**2**
II	**General instructions**	**3**
III	**Exercise 0: Puzzle solver**	**4**
IV	**Exercise 1: Puzzle solver advanced**	**6**
V	**Submission and peer-evaluation**	**8**

Chapter I

Preamble

Here is what Perplexity has to say about Game Theory:

Game theory in AI is a mathematical framework that models strategic interactions between rational agents, whether they are humans, computers, or other entities.
It provides tools for analyzing and predicting decision-making in competitive or cooperative scenarios.
In the context of AI, game theory is applied to develop algorithms for game-playing, autonomous vehicle navigation, multi-agent systems, and various real-world optimization problems.
By incorporating concepts like Nash equilibria, Pareto efficiency, and strategic planning, game theory enables AI systems to make intelligent decisions in complex environments where multiple actors influence outcomes123.
This interdisciplinary field combines elements of mathematics, economics, and computer science to create more sophisticated and adaptable AI systems capable of reasoning about strategic interactions.

But we will start by using Game Theory... in a game :)

Chapter II

General instructions

Unless explicitly specified, the following rules will apply to this project.

- This subject is the one and only trustable source. Don't trust any rumor.
- Your assignments will be evaluated by your peers.
- You have a question? Ask your left neighbor. Otherwise, try your luck with your right neighbor.
- You must read the examples thoroughly. They can reveal requirements that are not obvious in the assignment's description.
- Good luck and good work on this project!

Chapter III

Exercise 0: Puzzle solver

	Exercise : 00
Exercise 0: Puzzle solver	
Turn-in directory : $ex00/$	
Files to turn in : `puzzle_solver.py`	
Forbidden functions : `None`	

Do you know the 15-Puzzle game?
If not, we recommend that you quickly look it up on the internet to understand how the game works. If English is not your native language, we suggest you switch the Wikipedia page to your preferred language to better understand the terminology.

The goal of this exercise is to create a program that can solve a 3x3 sliding puzzle. The traditional 15-Puzzle is a 4x4 grid, but we will use the simpler version. Therefore, you will be solving an 8-Puzzle!
This program will allow you to understand how the basic algorithm used to solve these puzzles works.

`Generate and resolve the 3x3 sliding puzzle:`

- You must generate a random 3x3 initial puzzle.
- For evaluation purposes, it is possible to force a specific initial state through the command line. You can choose your preferred method to force the initial state, such as reading from a file or using command line parameters.
- You must use an algorithm that simulates the allowed moves of the 8 sliding cells.
- You will display the first solution found that solves the puzzle.
- You must handle the case where the puzzle is unsolvable.

`The User Interface must include the following features:`

- You must display the initial state of the puzzle.

- Display the series of steps you found to solve the puzzle.
- If the puzzle is unsolvable, display a message indicating that the puzzle is unsolvable.

`Example Execution for solvable puzzle`:

```
python3 puzzle_solver.py
Initial state of the puzzle:
1 2 3
4 0 6
7 5 8

Solving the puzzle...
Solution found in 2 steps.

Step 1:
1 2 3
4 5 6
7 0 8

Step 2:
1 2 3
4 5 6
7 8 0
```

`Example Execution for unsolvable puzzle`:

```
python3 puzzle_solver.py
Initial state of the puzzle:
1 2 3
4 5 6
8 7 0

Solving the puzzle...

Puzzle is unsolvable!
```

Chapter IV

Exercise 1: Puzzle solver advanced

	Exercise : 01
Exercise 1: Puzzle solver advanced	
Turn-in directory : $ex01/$	
Files to turn in : `puzzle_solver_ad.py`	
Forbidden functions : `None`	

We are still playing the same game! We will continue with our 3x3 sliding puzzle. However, this time, you will need to explore all possible paths to find the shortest one.

`Generate and resolve the 3x3 sliding puzzle:`

- You must generate a random 3x3 initial puzzle.
- For evaluation purposes, it is possible to force a specific initial state through the command line. You can choose the method to force the initial state such as reading from a file or using command line parameters.
- You must use an algorithm that tests all possible moves and all possible positions of the 8 sliding cells.
- You may find multiple solutions to solve the puzzle; you need to identify the one that requires the fewest moves.
- You must handle the case where the puzzle is unsolvable, as mentioned previously.

`The User Interface must include the following features:`

- You must display the initial state of the puzzle.
- Display the number of moves required to solve the puzzle, each time you find a different path.
- Display the steps of the shortest path found to solve the puzzle.
- If the puzzle is unsolvable, display a message indicating that the puzzle is unsolvable.

Example Execution for solvable puzzle:

```
python3 puzzle_solver_ad.py
Initial state of the puzzle:
1 2 3
4 0 6
7 5 8

Solving the puzzle...
Solving in 10 moves
Solving in 2 moves

Shortest path found: 2 moves

Step 1:
1 2 3
4 5 6
7 0 8

Step 2:
1 2 3
4 5 6
7 8 0
```

Example Execution for unsolvable puzzle:

```
python3 puzzle_solver_ad.py
Initial state of the puzzle:
1 2 3
4 5 6
8 7 0

Solving the puzzle...

Puzzle is unsolvable!
```

Chapter V

Submission and peer-evaluation

- Create an `ai-ml_discovery` folder at the root of your home, and move around in it.

- Create a new module folder with this format:

```
module[number_of_the_module]
example: module3 (for module 3)
```

  and navigate to it.

- From now on, all exercises should be in the correct folder rendering (exercise of module 0, in the module0 folder, exercise of module 1, in the module1 folder, etc.). for example:

```
[ai-ml_discovery/module0/[exercise_name].py]
[ai-ml_discovery/module1/[exercise_name].py]
etc...
```



Please note, during your defense anything that is not present in the folder dedicated to the project will not be checked.